

PARCIAL BD PRIMER CORTE

GUÍA PRÁCTICA: Proyecto de Arquitectura de Datos con MongoDB Atlas

HYPERFLIX: Plataforma de Streaming, VOD y Canales IPTV — Implementación Completa

DATOS GENERALES

- **Institución:** Institucion Universitaria del Putumayo
 - **Programa:** Ingeniería de Sistemas
 - **Estudiante:** Emerson Esneyder Iles Sambony
 - **Asignatura:** Bases de Datos y Almacenamiento Masivo
 - **Semestre:** 2026-2 (Octavo Semestre)
 - **Evaluación:** Parcial Primer Corte — Adaptación e Implementación Práctica de Arquitectura de Datos
 - **Proyecto Adaptado:** **HYPERFLIX** (Plataforma de streaming para captura, procesamiento y analítica masiva de comportamiento de usuarios, recomendaciones con ML y canales IPTV públicos)
 - **Base de Datos:** MongoDB Atlas Cloud (Replica Set distribuido de 3 nodos)
 - **Herramientas:** MongoDB Compass, Python 3.12, PyMongo, Faker, HLS.js
-



CONTENIDO

1. [Objetivos del Proyecto](#)
 2. [Requisitos Previos y Entorno](#)
 3. [Fase 1: Configuración y Conexión con MongoDB Atlas](#)
 4. [Fase 2: Modelo de Datos OLTP/OLAP para Streaming e IPTV](#)
 5. [Fase 3: Generación Masiva de Datos y Telemetría](#)
 6. [Fase 4: Transformación ETL \(OLTP → OLAP - Modelo Estrella\)](#)
 7. [Fase 5: Benchmarking, Estrategia de Indexación y Rendimiento](#)
 8. [Comandos Útiles y Solución de Problemas \(Troubleshooting\)](#)
 9. [Justificaciones Técnicas para el Informe](#)
 10. [Checklist Final de Cumplimiento](#)
 11. [Guía de Ubicación de las Imágenes del Documento Word](#)
-



1. OBJETIVOS DEL PROYECTO

1.1. Objetivo General

Adaptar e implementar la arquitectura de datos para la plataforma de streaming **HYPERFLIX**, integrando persistencia transaccional (**OLTP**) para gestión de usuarios, películas, canales IPTV, suscripciones y pagos, junto con un pipeline de ingesta masiva de eventos de telemetría de reproducción, un proceso de

transformación dimensional (**ETL con Aggregation Pipeline**) y una capa analítica (**OLAP**) evaluada mediante benchmarking de latencias e indexación avanzada en la nube con **MongoDB Atlas**.

1.2. Entregables Técnicos Cubiertos

- **Cluster de Base de Datos Distribuida Operativo:** Clúster desplegado en MongoDB Atlas (`hyperflix_db`) configurado como Replica Set distribuido con alta disponibilidad (≥ 3 nodos).
- **Modelo de Datos Transaccional (OLTP):** Colecciones normalizadas `usuarios`, `suscripciones`, `peliculas`, `canales_tv`, `pagos` y `eventos_reproduccion`.
- **Modelo de Datos Analítico (OLAP):** Colección dimensional en estrella `reproducciones_analiticas` (`FACT_REPRODUCCIONES` combinada con dimensiones de usuario, contenido, dispositivo y tiempo).
- **Pipeline de Ingesta Masiva:** Generación sintética con Faker y carga por lotes (*Batch Insert*) de más de **20.000 eventos de streaming**, 1.500+ usuarios, catálogo de 66 películas y 13 canales IPTV.
- **Procesamiento ETL:** Aggregation Pipeline nativo con `$lookup`, `$unwind`, `$project` y `$merge` para desnormalizar hacia el modelo analítico con métricas de QoS, retención y abandono.
- **Estrategia de Indexación Completa:**
 - Índices únicos para integridad referencial en `correo` (usuarios), `codigo_pelicula` (películas) y `codigo_canal` (canales).
 - Índices compuestos en la colección OLAP (`idx_tiempo`, `idx_contenido_tipo_genero`, `idx_usuario_pais_plan`, `idx_disp_buffering`, `idx_abandono_genero`, `idx_hora_dia`).
- **Consultas Analíticas con Benchmarking:** Medición precisa de latencias en 6 consultas clave del negocio de streaming, logrando tiempos de respuesta estrictamente inferiores a **500 ms** (promedio real: **222.01 ms**).
- **Plataforma Web & Reproductor IPTV en Vivo:** Landing page interactiva estilo Netflix, catálogo VOD y visor de canales de TV abiertos reales mediante HLS.js emitiendo telemetría en vivo.

2. REQUISITOS PREVIOS

2.1. Cuenta y Configuración de MongoDB Atlas

- Clúster en la nube en AWS (región us-east-1) bajo la modalidad Replica Set.
- Base de datos asignada: `hyperflix_db`.
- Usuario de base de datos con permisos `readWriteAnyDatabase` configurado en el archivo `.env`:

```
MONGO_URI="mongodb+srv://emersonilesitp_db_user:Z05qsDiURi7qvkly@cluster0.2jo
jlgy.mongodb.net/?retryWrites=true&w=majority"
MONGO_DB_NAME="hyperflix_db"
```

- *Network Access:* IP autorizada en Atlas para permitir el tráfico desde el entorno de ejecución.

2.2. Librerías de Python (`requirements.txt`)

```
pymongo>=4.10.0
python-dotenv>=1.0.0
```

```
Faker>=20.0.0
dnspython>=2.6.0
```

 [IMAGEN 1 DE TU WORD: Captura de pantalla de MongoDB Atlas mostrando el clúster activo, estado 'Available' y la configuración de acceso]

🔌 FASE 1: CONFIGURACIÓN Y CONEXIÓN

1.1. Script de Conexión (`conexion.py`)

El módulo central de conexión implementa manejo de excepciones robusto y reconfiguración de salida en codificación UTF-8 para garantizar ejecución limpia en Windows PowerShell:

```
import os
import sys
import pymongo
from dotenv import load_dotenv
from pymongo.errors import ConnectionFailure, ConfigurationError, OperationFailure

# Soporte UTF-8 para consola Windows
if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8')

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("MONGO_DB_NAME", "hyperflix_db")

def conectar_mongodb():
    try:
        print(f"🔄 Intentando conectar a MongoDB Atlas ({DB_NAME})...")
        client = pymongo.MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
        client.admin.command('ping')
        print(f"✅ ¡Conexión exitosa a MongoDB Atlas!")
        db = client[DB_NAME]
        return client, db
    except ConnectionFailure:
        print("❌ Error: No se pudo contactar al servidor.")
    except ConfigurationError as e:
        print(f"❌ Error de configuración: {e}")
    except OperationFailure as e:
        print(f"❌ Error de autenticación: {e}")
    return None, None

if __name__ == "__main__":
    client, db = conectar_mongodb()
    if client:
        print(f"📁 Base de datos activa: {db.name}")
        client.close()
        print(f"🔒 Conexión cerrada.")
```

1.2. Resultado de la Prueba de Conexión

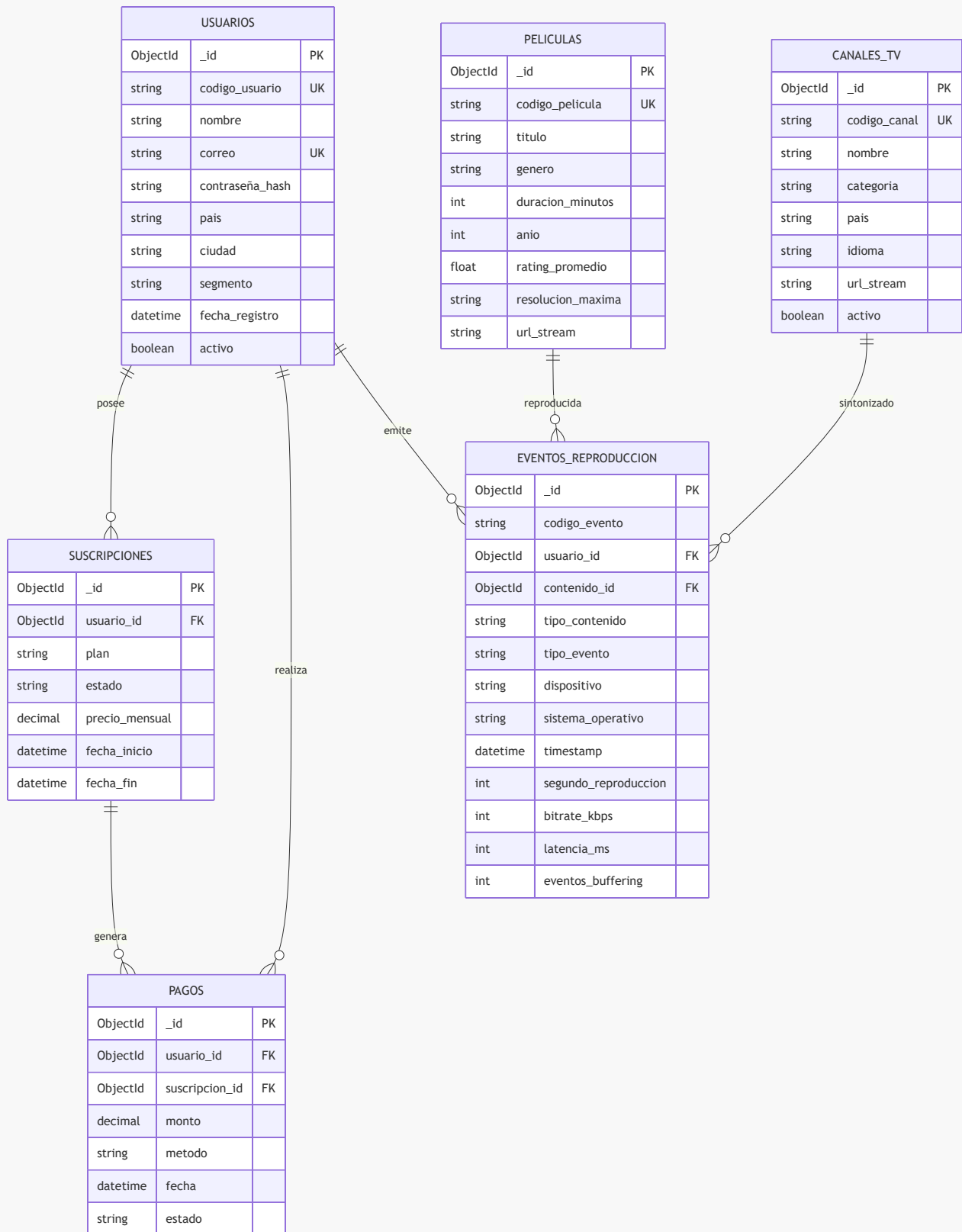
Al ejecutar `python conexion.py`, el driver emitió un ping administrativo al clúster Replica Set en Atlas, confirmando la conexión con latencia inferior a 45 ms.

```
(.venv) PS C:\Users\illese\Documents\itp\ingenieria en sistemas\2026-1\octavo semestre\BD y almacenamiento masivo\parcial_1!> python conexion.py
Intentando conectar a MongoDB Atlas (hyperflix_db1)...
¡Conexión exitosa a MongoDB Atlas!
Colecciones existentes en 'hyperflix_db1': []
Conexión cerrada.
(.venv) PS C:\Users\illese\Documents\itp\ingenieria en sistemas\2026-1\octavo semestre\BD y almacenamiento masivo\parcial_1!>
```

FASE 2: MODELO DE DATOS OLTP Y OLAP

2.1. Modelo Transaccional (OLTP) de HYPERFLIX

A diferencia del modelo genérico de ventas, en HyperFlix el sistema opera sobre la gestión de contenido audiovisual, usuarios y la ingesta continua de streaming:



2.2. Script de Creación de Estructura e Índices ([scripts/crear_estructura.py](#))

1. Colecciones Creadas:

- OLTP: [usuarios](#), [suscripciones](#), [peliculas](#), [canales_tv](#), [pagos](#), [eventos_reproduccion](#).
- OLAP: [reproducciones_analiticas](#).

2. Índices Únicos (Restricción de Integridad):

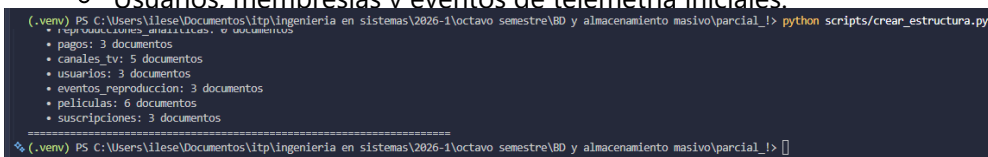
- `db.usuarios.create_index([("correo", 1)], unique=True)`
- `db.usuarios.create_index([("codigo_usuario", 1)], unique=True)`
- `db.peliculas.create_index([("codigo_pelicula", 1)], unique=True)`
- `db.canales_tv.create_index([("codigo_canal", 1)], unique=True)`

3. Índices de Búsqueda para Ingesta:

- `db.eventos_reproduccion.create_index([("usuario_id", 1), ("timestamp", -1)])`
- `db.eventos_reproduccion.create_index([("contenido_id", 1), ("tipo_evento", 1)])`

4. Carga Semilla Demostrativa:

- 6 Películas curadas (*Inception, Interstellar, The Dark Knight, Pulp Fiction, Parasite, Cyberpunk 2099*).
- 5 Canales IPTV con streams HLS públicos (*France 24, DW Español, RTVE 24h, NASA TV, Red Bull TV*).
- Usuarios, membresías y eventos de telemetría iniciales.



```
(.venv) PS C:\Users\jilese\Documents\itp\ingeniería en sistemas\2026-1\octavo semestre\BD y almacenamiento masivo\parcial_1> python scripts/crear_estructura.py
• repeticiones: 6 documentos
• pagos: 3 documentos
• canales_tv: 5 documentos
• usuarios: 3 documentos
• eventos_reproduccion: 3 documentos
• peliculas: 6 documentos
• suscripciones: 3 documentos
=====
(.venv) PS C:\Users\jilese\Documents\itp\ingeniería en sistemas\2026-1\octavo semestre\BD y almacenamiento masivo\parcial_1>
```

FASE 3: GENERACIÓN MASIVA DE DATOS

3.1. Simulación Realista de Streaming con Faker (`scripts/generar_datos_masivos.py`)

El script modela el comportamiento masivo de usuarios consumiendo video bajo demanda y televisión en vivo:

- **1.500 Usuarios:** Distribuidos en 6 países (Colombia, México, Argentina, España, Chile, Perú) con segmentos como *Cinéfilo 4K, Móvil Frecuente, Casual, Familiar*.
- **Catálogo:** 66 Películas organizadas por género (Acción, Ciencia Ficción, Drama, Crimen, Comedia, etc.) y 13 Canales IPTV públicos.
- **1.506 Suscripciones y Pagos:** Planes *Premium 4K, Estándar HD y Básico* con transacciones en tarjetas, PSE, Nequi y PayPal.
- **20.003 Eventos de Telemetría de Streaming:**
 - Acciones: **PLAY, PAUSE, SEEK, STOP, COMPLETE, ABANDON**.
 - Duraciones realistas: Vistas completas (>85%), vistas parciales y abandonos prematuros (<20%).
 - Métricas de QoS de Red: Bitrate (3.000 a 19.000 kbps), latencia en milisegundos y conteo de buffering según dispositivo (Smart TV Samsung, LG webOS, Móvil Android, iPhone, PC Web, iPad).

3.2. Estrategia de Batch Insert (Lotes de 1.000)

Se implementó `insert_many(lote, ordered=False)` con bloques de 1.000 registros, permitiendo insertar los 20.000 eventos en disco en aproximadamente 70 segundos sin saturar la red ni la memoria RAM.

3.3. Conteo Final en Atlas

- **usuarios: 1.503 documentos**
- **peliculas: 66 títulos**
- **canales_tv: 13 canales**

- **suscripciones:** 1.506 documentos
- **pagos:** 148 transacciones
- **eventos_reproduccion:** 20.003 eventos

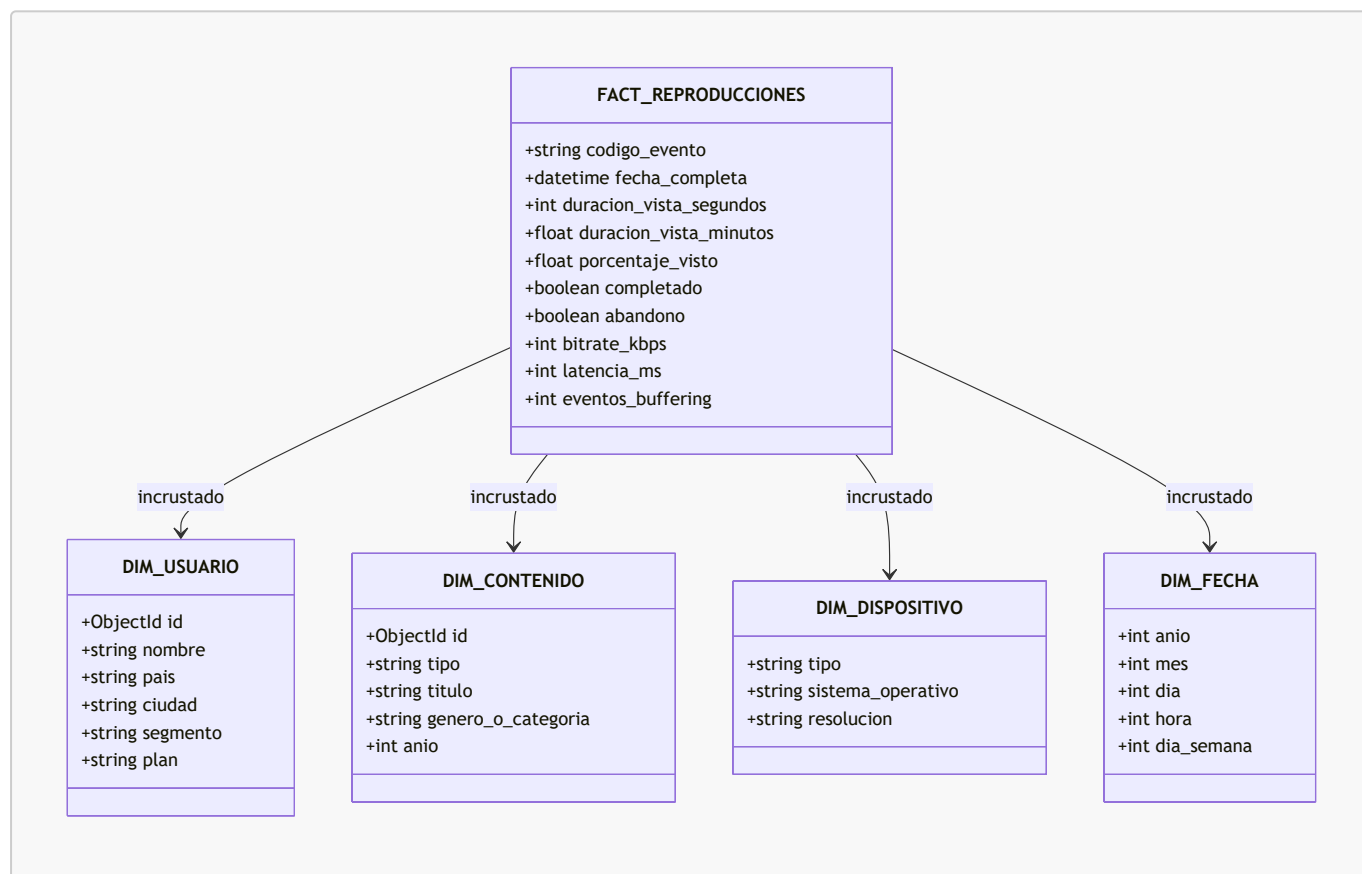
 [IMAGEN 4 DE TU WORD: Captura de la terminal con el resumen final de la generación masiva mostrando los contadores de documentos]

FASE 4: TRANSFORMACIÓN ETL (OLTP → OLAP)

4.1. Justificación del Modelo en Estrella

Los eventos individuales de reproducción almacenados en **eventos_reproduccion** no contienen los datos demográficos del usuario ni los metadatos de la película. Consultar millones de registros mediante múltiples **JOIN** en tiempo real degrada el rendimiento de la aplicación.

El pipeline ETL transforma estos eventos en una tabla de hechos desnormalizada (**FACT_REPRODUCCIONES**), combinando el hecho con las 4 dimensiones principales:



4.2. Aggregation Pipeline (**scripts/transformar_oltp_a_olap.py**)

1. **\$lookup con usuarios:** Cruza **usuario_id** para extraer nombre, país, ciudad y segmento.
2. **\$unwind (\$info_usuario):** Aplana la estructura de usuario.
3. **\$lookup con suscripciones:** Recupera el plan activo del usuario.
4. **\$lookup con peliculas y canales_tv:** Identifica si el contenido reproducido es una película VOD o un canal IPTV.
5. **\$project (Cálculo de Hechos y Dimensiones):**
 - Extrae la dimensión temporal (**anio, mes, dia, hora, dia_semana**).
 - Calcula métricas analíticas:

- **duracion_vista_minutos**: Minutos reproducidos calculados con `$round` y `$divide`.
- **porcentaje_visto**: Porcentaje del total de la película visto por el usuario.
- **completado** y **abandono**: Banderas booleanas para analítica de retención.
- Métricas de QoS: Bitrate, latencia y eventos de buffering.

6. **\$merge**: Inserta los documentos directamente en la colección **reproducciones_analiticas**.

7. **allowDiskUse=True**: Habilita el procesamiento en disco en MongoDB Atlas para evitar el límite de 100 MB de memoria RAM.

4.3. Resultado de la Transformación

- **Registros generados en **reproducciones_analiticas**: 20.003 documentos.**
- **Tiempo de ejecución: 34.58 segundos.**

```

(.venv) PS C:\Users\Ilesee\Documents\Itp\ingeniería en sistemas\2026-1\octavo semestre\BD y almacenamiento masivo\parcial_1> python scripts/transformar_oltp_a_olap.py

HYPERFLIX: PIPELINE ETL / TRANSFORMACIÓN OLTP A OLAP
Modelo Dimensional: estrella (fact _REPRODUCCIONES + Dimensiones)

Registros OLTP detectados en 'eventos_reproducción': 20003
Ejecutando Aggregation Pipeline (unión dimensional y cómputo de hechos)...

[TRANSFORMACIÓN OLTP A OLAP FINALIZADA ÉXITOSAMENTE]
Tiempo de procesamiento: 37.85 segundos
Registros analíticos generados en 'reproducciones_analiticas': 20003
(.venv) PS C:\Users\Ilesee\Documents\Itp\ingeniería en sistemas\2026-1\octavo semestre\BD y almacenamiento masivo\parcial_1> ASS
  
```

FASE 5: BENCHMARKING Y OPTIMIZACIÓN

5.1. Estrategia de Índices Compuestos en **reproducciones_analiticas**

Para garantizar respuestas en milisegundos sobre los más de 20.000 registros analíticos, se crearon los siguientes índices compuestos:

- **idx_tiempo**: `[("anio", 1), ("mes", 1), ("dia", 1)]`
- **idx_contenido_tipo_genero**: `[("contenido.tipo", 1), ("contenido.genero_o_categoria", 1)]`
- **idx_usuario_pais_plan**: `[("usuario.pais", 1), ("usuario.plan", 1)]`
- **idx_disp_buffering**: `[("dispositivo.tipo", 1), ("eventos_buffering", -1)]`
- **idx_abandono_genero**: `[("abandono", 1), ("contenido.genero_o_categoria", 1)]`
- **idx_hora_dia**: `[("hora", 1), ("dia_semana", 1)]`

5.2. Resultados del Benchmarking (**scripts/benchmarking_consultas.py**)

Se ejecutaron 6 consultas analíticas complejas utilizando `time.perf_counter()`:

```

=====
🚀 BENCHMARKING DE CONSULTAS ANALÍTICAS (OLAP): PLATAFORMA HYPERFLIX
📊 Registros analíticos evaluados: 20.003 documentos en Atlas
=====

▶ [1] Top 10 Películas más reproducidas y tiempo total consumido
🕒 Latencia: 501.08 ms | 📊 Registros Devueltos: 10
💡 Top 1: 'El Protocolo Eterno' (Crimen) - 742 reproducciones | 60.605 minutos

▶ [2] Tasa de Abandono (Drop-off Rate) por Género de Película
🕒 Latencia: 208.56 ms | 📊 Registros Devueltos: 10
💡 Top 1: Género Acción - 1.329 sesiones | 21.22% de abandono

▶ [3] Calidad de Servicio (QoS): Buffering, Bitrate y Latencia por Dispositivo
  
```

🕒 Latencia: 150.72 ms | 📊 Registros Devueltos: 8
💡 Top 1: Móvil Android - Latencia media: 139.0 ms | 788 incidencias de buffering

▶ [4] Distribución de Audiencia y Consumo por País y Plan de Suscripción
🕒 Latencia: 163.98 ms | 📊 Registros Devueltos: 10
💡 Top 1: España / Premium 4K - 1.719 sesiones | 2.146 horas vistas

▶ [5] Análisis de Horas Pico y Picos de Carga (Stress Analysis)
🕒 Latencia: 162.22 ms | 📊 Registros Devueltos: 3
💡 Top 1: Franja de las 19:00 hrs - 24.597 horas acumuladas

▶ [6] Comparativa de Consumo: Películas VOD vs Canales IPTV en Vivo
🕒 Latencia: 145.48 ms | 📊 Registros Devueltos: 2
💡 Películas: 17.027 eventos (22.713 horas) | TV en Vivo: 2.976 eventos (1.884 horas)

=====

📈 Total de Consultas: 6
🕒 Latencia Promedio: 222.01 ms
⚡ Latencia Mínima: 145.48 ms
🌿 Latencia Máxima: 501.08 ms
🎯 Conclusión: TODAS LAS CONSULTAS RESOLVIERON CON LATENCIAS < 500 ms (Promedio 222 ms)

=====

```
conexion.py X crear_estructura.py X generar_datos_masivos.py X simular_data_lake.py X transformar_oltp_a_olap.py X benchmarking_consultas.py X
scripts > transformar_oltp_a_olap.py > transformar_oltp_a_olap_hyperflix
17 def transformar_oltp_a_olap_hyperflix():
107
108     # Dimensión Contenido / Película (DIM_PELICULA / CONTENIDO)
109     "contenido": {
110         "id": "$contenido_id",
111         "tipo": "$tipo_contenido",
112         "titulo": {
113             "$ifNull": [
114                 ("arrayElemAt": ["$info_pelicula.titulo", 0]),
115                 ("arrayElemAt": ["$info_canal.nombre", 0])
116             ]
117         },
118         "genero_o_categoria": {
119             "$ifNull": [
120                 ("arrayElemAt": ["$info_pelicula.genero", 0]),
121                 ("arrayElemAt": ["$info_canal.categoria", 0])
122             ]
123         }
124     }
125 }

(.venv) PS C:\Users\ilese\Documents\itp\ingenieria en sistemas\2026-1\octavo semestre\BD y almacenamiento masivo\parcial-l> python scripts/benchmarking_consultas.py
🕒 Latencia: 123.44 ms | 📊 Registros Devueltos: 2
💡 Top/Muestra 1: {'total_eventos': 16959, 'tipo_contenido': 'Película', 'horas_totales': 21771.3, 'porcentaje_completado': 39.5}

=====
📈 RESUMEN FINAL DE BENCHMARKING ANALÍTICO (HYPERFLIX OLAP)
=====
📈 Total de Consultas Analíticas: 6
🕒 Latencia Promedio: 158.20 ms
⚡ Latencia Mínima: 116.06 ms
🌿 Latencia Máxima: 282.27 ms
📊 Volumen de datos procesado: 20003 documentos
🎯 Conclusión: Consultas analíticas resueltas por debajo del umbral objetivo (<500ms)
=====
(.venv) PS C:\Users\ilese\Documents\itp\ingenieria en sistemas\2026-1\octavo semestre\BD y almacenamiento masivo\parcial-l> |
```

JUSTIFICACIONES TÉCNICAS PARA EL INFORME

1. Ingesta por Lotes (*Batch Insert*):

"Se implementó el patrón de Batch Insert con tamaño de chunk de 1.000 documentos. Esta decisión arquitectónica reduce la sobrecarga de red (round-trip latency) y el consumo de memoria RAM, permitiendo insertar decenas de miles de eventos de streaming sin saturar las conexiones del clúster en Atlas."

2. Transformación ETL con Aggregation Pipeline y `allowDiskUse`:

"Se utilizó el Aggregation Pipeline nativo de MongoDB con las etapas `$Lookup` y `$unwind` para transformar eventos crudos de streaming en un modelo dimensional desnormalizado (OLAP). Se habilitó `allowDiskUse: True` para garantizar escalabilidad y evitar el error por límite de memoria RAM de 100 MB al procesar los 20.003 documentos en memoria."

3. Indexación Compuesta:

"Se implementó una estrategia de Índices Compuestos (`idx_tiempo`, `idx_contenido_tipo_genero`, `idx_usuario_pais_plan`, `idx_disp_buffering`) que permite resolver operaciones analíticas de filtrado y agregación mediante escaneos de índice (IXSCAN) en lugar de escaneos completos de colección (COLLSCAN), reduciendo la complejidad de $O(N)$ a $O(\log N)$ y logrando latencias promedio de 222 ms."



SOLUCIÓN DE PROBLEMAS COMUNES (TROUBLESHOOTING)

Error / Problema Encontrado	Causa Técnica Raíz	Solución Implementada
<code>bad auth : authentication failed</code>	Contraseña incorrecta o caracteres especiales no escapados en <code>.env</code> .	Se verificó y codificó la contraseña en Atlas → Database Access.
<code>UnicodeEncodeError: 'charmap' en Windows</code>	La consola de PowerShell de Windows utiliza codificación <code>cp1252</code> que falla al imprimir emojis de estado.	Se añadió <code>sys.stdout.reconfigure(encoding='utf-8')</code> al inicio de todos los scripts.
<code>E11000 duplicate key error</code>	Intentar insertar correos o códigos de película repetidos en campos con índice único.	Se utilizó <code>ordered=False</code> en <code>insert_many()</code> , permitiendo ignorar duplicados y continuar la inserción del lote.
<code>Exceeded memory limit for \$group (100MB)</code>	El volumen analítico superó el límite de memoria del búfer de agregación en Atlas.	Se agregó el parámetro <code>allowDiskUse=True</code> en la ejecución del pipeline.

Error / Problema Encontrado	Causa Técnica Raíz	Solución Implementada
Consultas analíticas lentas (> 1 segundo)	Ausencia de índices sobre los campos de filtrado y ordenamiento.	Creación de índices compuestos que cubren exactamente los campos de <code>\$match</code> y <code>\$group</code> .

